Next-Gen AI Compiler

Agents On The Control Plane

Agents Own Search · Orchestration · Synthesis.
Compilers Own Lowering · Legality · Measure · Fallback.

Ask: Is The Hybrid Prediction (∼2027–31) The Right Bet For Your Roadmap?

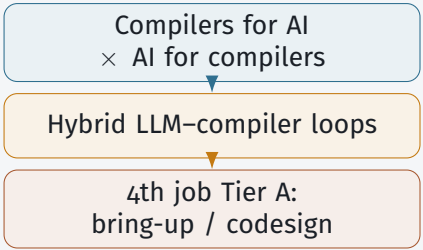
Expert Briefing · ∼25–30 min + Q&A · SURVEY §5 Primary

Agenda

1. Verdict + Four Jobs
2. Architecture Evolution
3. Prediction Checkpoints
4. Commercial Blockers (Top 5)
5. Gaps That Gate The Future
6. Trends · Stance · Ask

Executive verdict

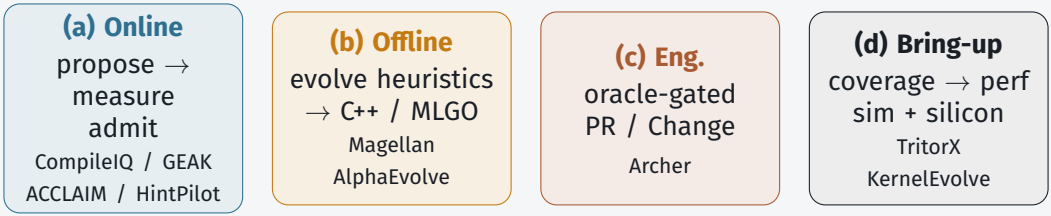
Agents reshape the control plane more than they replace the data plane.



Will not happen soon

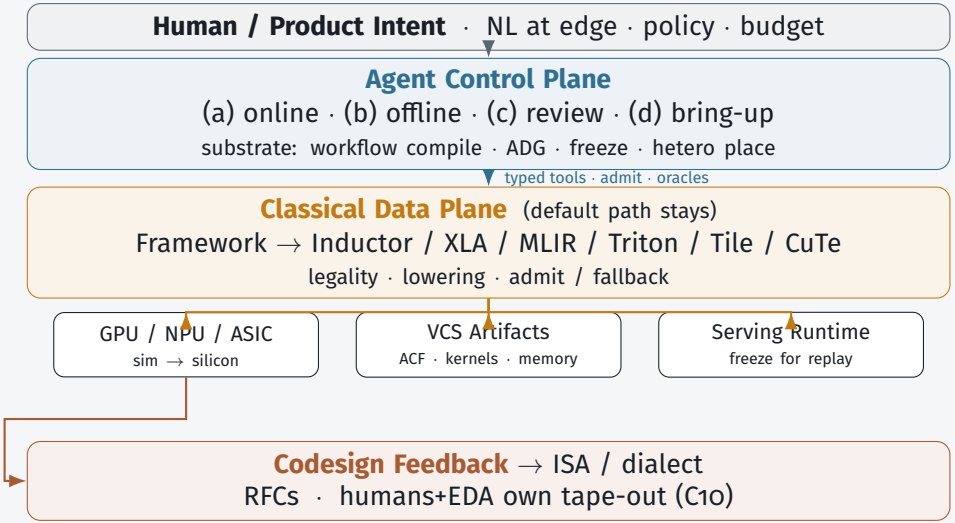
- Unconstrained LLM replaces Inductor / `opt`
- Autonomous chip tape-out via compiler agents (C10)

Four agent jobs — architecture spine

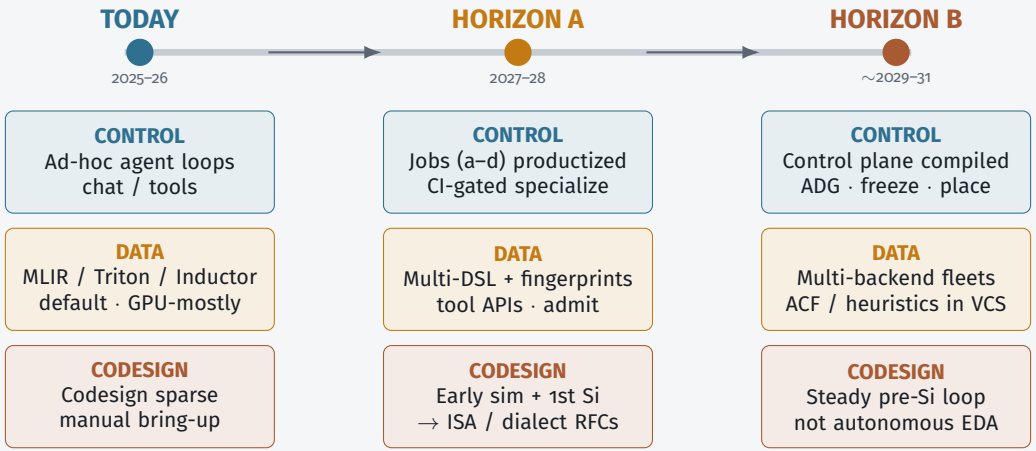


Control plane jobs sit *above* classical lowering — not instead of it.

Architecture — Target Stack (§5.1)



Architecture evolution — Today → A → B



Data plane never goes away — agent graph becomes compiled, audited, amortized.

What ships / does not by 2028

Predicted to ship

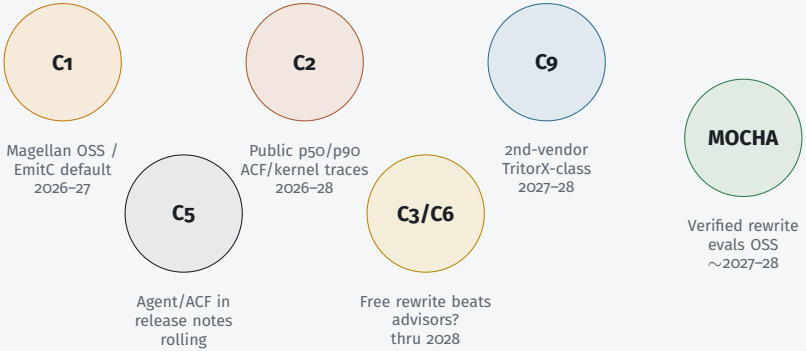
- Agent-addressable tool APIs
- Hot-path specialize (not silent default-all)
- Magellan *and* MLGO both live
- Oracle PR review in serious orgs
- Coverage-first ASIC bring-up
- Triton-family primary; multi-DSL rising

Does *not* ship

- Unconstrained LLM replaces `opt`/Inductor
- One agent IR for all vendors
- Kernel agents uniformly beat eager on fusion ladders
- Autonomous microarch tape-out

Horizon A success = CI-gated specialize + oracles + freeze artifacts

Checkpoints — when the prediction changes



Falsifiable sketch on C1–C10. We watch settlement signals — we do not average disagreements.

Checkpoint C1 — heuristics vs neural advisors

A — Magellan path

Evolve shippable C++ (EVOLVE-BLOCK)

Offline coding agent *is* the control-plane output

Product = evolutionary coding agent + review

B — MLGO / EmitC path

NN advisors stay in-tree

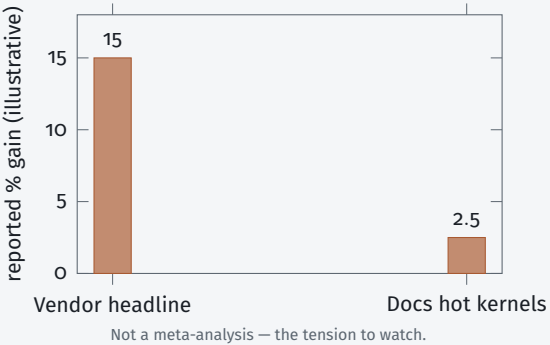
June 2026 PoR: internal inliner → Android/Fuchsia → Chrome multi-model

Agents train/feature NNs

Settlement: public Magellan llvm patches that displace MLGO *or* EmitC-MLGO becomes customer default

Watch LLVM syncs + OpenEvolve recipes quarterly through 2027

Checkpoints C2 + C5 — gains & default path



Not a meta-analysis — the tension to watch.

C2 "Agents become default" needs **p50 CI wins**, not best-kernel blogs.

C5 Online specialize (ACF/hints) vs offline eng (Magellan/MLGO) — both may live; *which is the default flag?*

Settlement: pinned public traces + release notes listing agent/ACF workflows.

Checkpoints C3 / C6 / C9 / C10 — API width & scope

C3 — free rewrite vs advisory
Flip: shared suite where free rewrite wins
⇒ wide rewrite API vs narrow ACF/hints

C6 — replace vs control plane
Flip: production default with *no* admit
⇒ rewrites survey baseline (now C6-B)

C9 — coverage vs peak
Flip: TritorX → KernelEvolve ladder public
⇒ SKU: coverage SLA then perf SLA

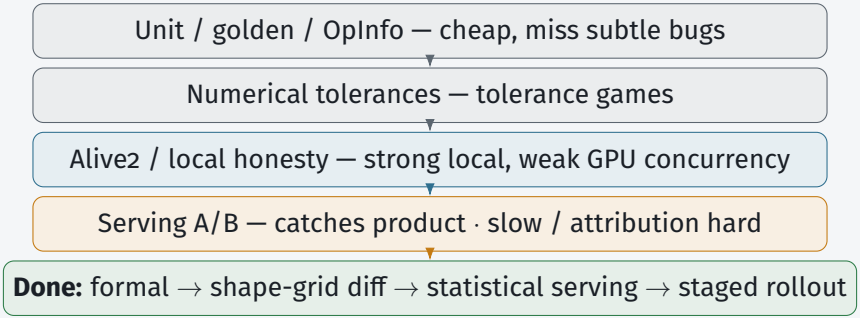
C10 — codesign vs auto tape-out
Flip: microarch primarily agent-proposed *and* compiler-oracle validated ⇒ enters EDA

Commercial blockers — top 5

1. **Oracles strong enough for money**
   Fast-but-wrong · liability
2. **Cost / replay / when agents run**
   Nondeterministic $N compiles
3. **Agent↔compiler contract**
   NL-only = demo
4. **Distributional production evidence**
   No default-path A/B
5. **Ownership · supply chain · HITL**
   Unowned agent code in TCB

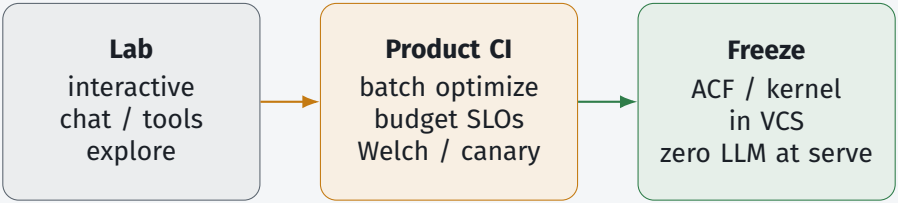
Each blocker gets one slide. Survey lean from §5.7 + §4.

Blocker 1 — Oracles for money



Room question: who owns false negatives when admit passes and production miscompiles?

Blocker 2 — Cost, replay, when may the agent run?



Cache key: (IR hash, HW, compiler ver, agent policy) · budget $/%gain

Falsifies commercially: “chat with compiler on every build” without spend/latency SLOs.

Blocker 3 — Agent↔compiler contract

A. NL + pasted logs — demo only

B. Structured admit traces — CI/SBOM

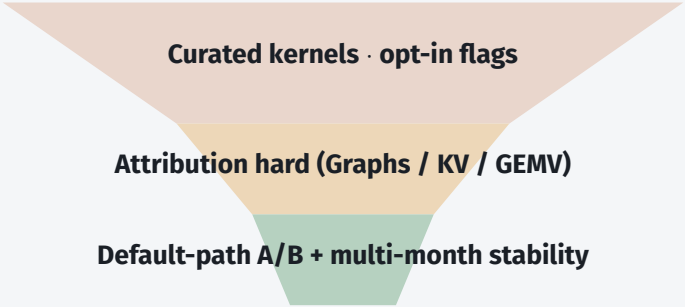
C. Typed tool APIs — required action space

D. Hybrid — NL view over traces

```
admit_record = {graph_hash, hw_id, compiler_ver, action[], oracle[],
artifact_digest, policy_id}
```

Lean: **C + B**; NL is a view, not source of truth. ACF = Advanced Control File.

Blocker 4 — Distributional production evidence



Marketing bar: p50/p90 + cost-to-compile · MLGO-style persistent QPS, not single-kernel headlines

Blocker 5 — Ownership, security, HITL

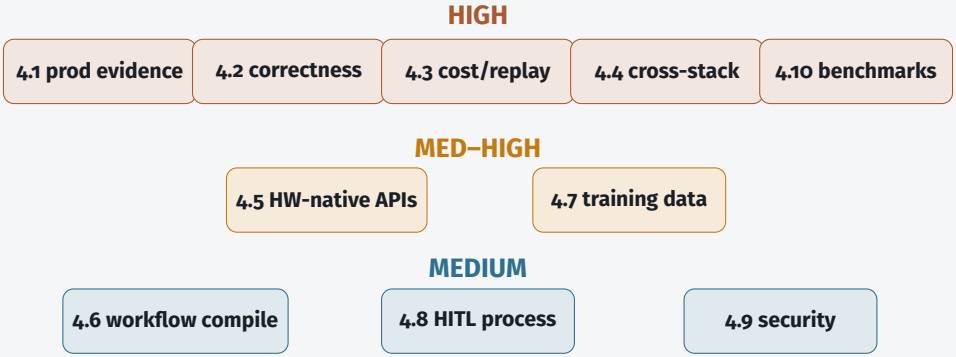
TCB risk
agent kernels / heuristics enter trusted base

Ownership
CODEOWNERS
signed provenance
sandbox

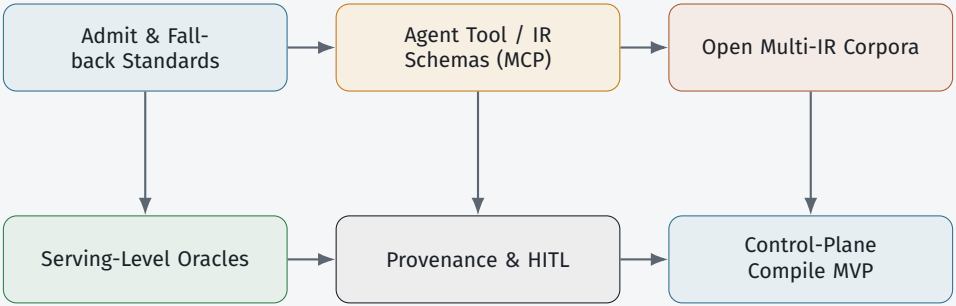
HITL capacity
agents ↑ drafts
reviewers = bottleneck

Lean: human CODEOWNER + signed admit + sandbox · oracle auto-merge only for **narrow** action classes
Generic forge AI ≠ compiler-oracle review (C7)

Gap map — blockers to §5, not a wishlist



Cross-Cutting Research Agenda



Near-Term: Specs That Make Hybrid Pipelines CI-Regressable.

Org Adoption Questions

1. Online (CompileIQ) vs Offline (Magellan) — which matches your release model?

2. Oracle stack: Alive2 / golden / serving A/B — who owns false negatives?

3. Agent contract surface: LLVM / MLIR / Triton / StableHLO / Tile?

4. How do you cache/regress traces across compiler *and* model upgrades?

5. Max $/build for a median X% win? Named maintainer per admitted artifact?

Trend backdrop — two stacks converging

Compilers for AI
mature · fragmented substrate

AI for compilers
rapid hybrid growth

Next-gen ≠ throw away LLVM/MLIR — make them **agent-addressable**.

Six active trends

A Hybrid guidance
free IR rewrite fragile
constrain action space

B RL → agents
tools · heuristics
workflow-compile

C MLIR + Triton
peak often still
vendor libs / Tile

D Kernel agents
industrial · Ker-
nelBench hard

E Verify in loop
local formal ≫
whole-program GPU/FP

F Broader object
FMware · mid-decode
agents as eng

Keep substrate, change control plane

Preserve

deterministic lowering
library kernels
CI regressability
local formal checks

Adopt

advisory search
multi-agent orchestration
heuristic synthesis
profile / verifier feedback

Anti-pattern: replace `opt`/Inductor with unconstrained LLM codegen

Working stance until conflicts settle

1. Hybrid control / data plane (C6-B)
2. Magellan || MLGO — parallel bets (C1)
3. Discount single-number speedups (C2)
4. Constrained actions + strong oracles (C3)
5. Demote generic SCM AI (C7)
6. Coverage→perf ladder; no autonomous EDA (C9/C10)

Claim status snapshot

Full claim map: SURVEY §7.

Systems gallery — mechanism clusters

Pass / heuristic
LLM Compiler · Compiler-R1 · Magellan
MLGO · HintPilot · ACCLAIM

Kernel / DSL
GEAK · KernelLLM · AgentCompile
CompileIQ · CuTeGen · AutoKernel ·
KForge

Bring-up / codesign
TritorX · KernelEvolve
Ascend hierarchical diagnosis

Boundary / negative
mlirAgent — below identity
on free IR rewrite

Prefer mechanisms over headline speedups.

Commercial checklist — handout

Architecture

typed tools + admit traces

VCS ≫ dense ≫ scratch-pad

FSM / plan for SLA

freeze before default-on

Trust

layered oracles

CODEOWNERS + prove-nance

joint version pins

A/B before “prod default”

Business

sell regressable artifacts + CI quota

customer owns outputs

coverage then perf SLA

token budget per %gain

Discussion

Thank you — hybrid bet stays the ask

1. Which checkpoint flips your investment first — C1, C2, or C9?
2. Harder commercial bar: **oracle strength** or **replay/freeze**?
3. Building job (a), (b), or (d) first — what do you refuse online?

Primary narrative: docs/SURVEY.md §5 → §4 → §1 · Backup: C1–C10 · P9–P23 · §9